CSE 4360 / 5364 - Autonomous Robots

Project 2: Manipulator Motion Planning

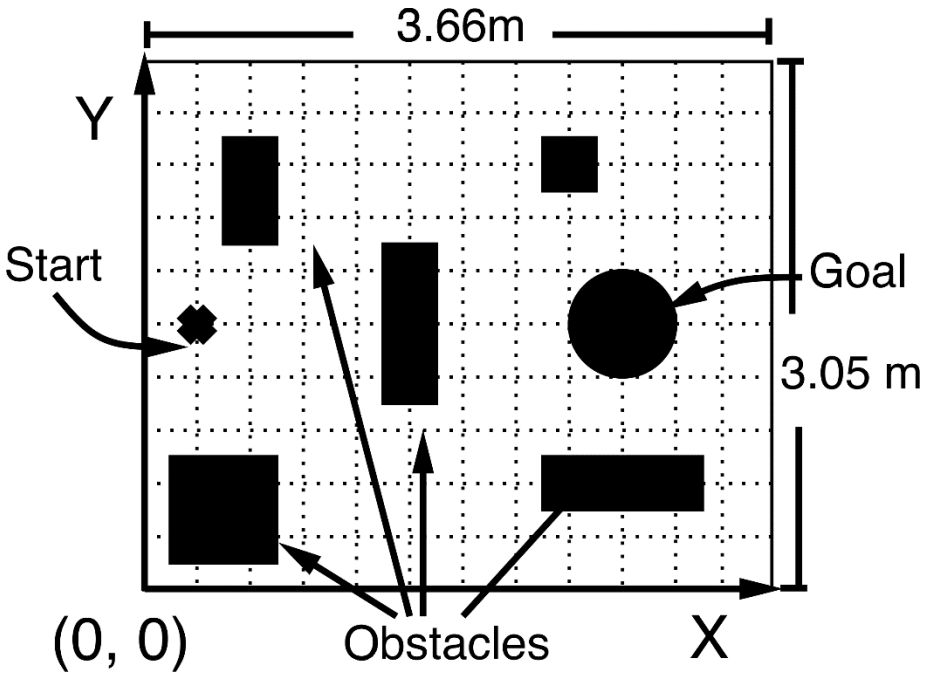
Due Date: _____

Objective: plan and execute a collision-free arm motion that grasps an object, carries it around virtual obstacles, and places it at a specified goal.

Overview

The objective of this assignment is to grasp and transport an object through a known obstacle course to a goal location. The initial object location, goal location, and obstacle geometry will be provided before the robot begins. Your program must use a motion-planning method discussed in class to determine an appropriate path and then execute that path with the robot arm. An illustration is shown below.

This project builds directly on Lab 1. You may use your Lab 1 code for commanding the arm and gripper. The new challenge is to determine where the robot should move rather than simply executing a predetermined sequence of arm commands.



Workspace and Virtual Obstacles

A paper coordinate grid will be placed on the table in front of the robot. The grid provides a convenient physical reference for specifying object, goal, and obstacle locations. It is not intended to define an occupancy-grid planning problem.

Obstacles will be virtual. Regions of the paper workspace will be identified as forbidden areas, and the grasped object must be transported around these regions. The obstacles may be specified by grid coordinates or by geometric boundaries such as rectangles or polygons.

The pick-up and placement portions of the task will require vertical motion. During the transport portion, the object should remain near a specified transport height. Virtual obstacles should be treated as extending through the permitted transport region, so simply lifting the object over an obstacle is not a valid solution.

Problem Specification

For each test configuration, you will be given:

- the initial location of the object;
- the goal location where the object must be placed;
- the locations and geometry of the virtual obstacles; and
- any additional parameters needed to interpret the workspace coordinates.

The object start, goal, and obstacle locations will be provided at compile/run time or immediately before testing. Test configurations will be chosen so that a feasible path exists. Your program should not contain a motion sequence that is hard-coded for a particular demonstration layout.

Encoding the Test Configuration

Given the start, goal, and obstacle locations, you will need to encode them in a form that your program can process. For example, in Python, rectangular obstacles could be represented by the coordinates of their lower-left and upper-right corners. A test configuration might be entered as follows:

```
# All dimensions are in meters.
# Each obstacle is stored as ((x_min, y_min), (x_max, y_max)).

MAX_OBSTACLES = 25

obstacles = [
   ((0.20, 0.15), (0.30, 0.35)),
   ((0.40, 0.30), (0.55, 0.40)),
   ((0.62, 0.10), (0.72, 0.28)),
]

object_start = (0.12, 0.12)
goal = (0.78, 0.48)
object_radius = 0.025
transport_height = 0.10
```

The precise data structure is up to you. Obstacles may instead be represented as polygons, line segments, configuration-space boundaries, or another geometric form appropriate for your planning method. You will be given a short period at the beginning of the demonstration to enter or load the specific test configuration.

The Project

1. Use the Robot Arm and Gripper

Begin with the manipulator control developed in Lab 1. The robot must approach the object, grasp it securely, lift it to the transport height, execute the planned transport motion, lower the object at the goal, and release it.

2. Implement a Motion-Planning Strategy

Implement a motion-planning strategy, based on methods discussed in class, that can generate a path for arbitrary valid start, goal, and obstacle configurations. Your planner should produce a path that carries the object from its initial location to the goal without entering a forbidden obstacle region and while remaining within the assigned workspace. You may not simply run A* on the occupancy grid. Select and implement an appropriate continuous-space or configuration-space planning approach discussed in class or the textbook.

3. Execute the Planned Motion

Convert the path produced by your planner into commands for the robot arm. The executed robot motion should follow the planned route closely enough that the transported object remains outside the virtual obstacles. You may use intermediate waypoints, inverse kinematics, or other techniques developed in the laboratory and course.

Object Size and Clearance

The transported object has nonzero size. Your planning method must therefore provide sufficient clearance for the object to pass around obstacles. A trajectory is not valid simply because a single point at the gripper or object center

remains outside an obstacle. You may account for object dimensions using an appropriate configuration-space construction or another method consistent with the techniques discussed in class.

Demonstration and Testing

During the project demonstration, the instructor or TA will specify the start and stop locations and provide one or more obstacle configurations. You will have a short period to enter or load the configuration before running your program. The obstacle configuration, object start, and goal will not be known in advance.

For each test, the robot should:

1. grasp the object at the specified starting location;
2. compute a path using your motion-planning algorithm;
3. transport the object around all virtual obstacles;
4. remain within the allowed workspace; and
5. place and release the object at the specified goal.

Report and Code

Each group must submit a short project report and the robot code. The report should describe the motion-planning method, how the environment and object geometry are supplied to the planner, how the planned path is converted to robot-arm motion, and any important implementation choices or limitations. Include sufficient discussion and supporting figures or results to demonstrate that you understand the planning method and its application to this problem. Include portions of code when appropriate. You must also include a short section or table on what each team member did over the course of the project.

Team members must be able to describe and answer questions about their code during your demonstration.

Suggested Grading

Component	Points
Motion-planning code/ algorithm and implementation	25
Execution	
Successful Pickup	5
Avoidance of virtual obstacles	25
Successful placement at the goal	5
Discussion of Code with TA/Instr.	20
Report	20
Total	**100*.4 for 40 max points**

I used Microsoft 365 Copilot to help revise and develop this assignment from an existing laboratory exercise that involved mobile robots.